

TITLE OF THE PROJECT

**BANK
MANAGEMENT
SYSTEM**

I developed a desktop application using Java Swing and AWT for the user interface, integrated with SQL for efficient data management.

The project focuses on creating a responsive and user-friendly system with smooth functionality. It demonstrates my skills in Java development, GUI design, and database handling.

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled **Bank Management System** submitted to **Axis Institute of Higher Education, Kanpur** in partial fulfillment of the requirement of the award of the degree of **Bachelor of Computer Applications (BCA)**, is original worked carried out by **Raj Singh** with **Roll No. 23071001273** under my guidance.

The matter embodied in this project is genuine work done by the student and has not been submitted weather to this University or to other University / Institute for the fulfillment of the requirement of any course of study.

Signature of the Student

(Roll No. 23071001273)

Date:

Signature of the Guide

(Assistance Professor)

Date:

STUDENT DECLARATION

I hereby declare that the project work entitled “**Bank Management System**” in partial fulfillment of the requirement for the award of degree of **BCA** and submitted to **CHHATRAPATI SHAHU JI MAHARAJ UNIVERSITY, KANPUR** is authentic record of my own work carried out during the period from 1th Jan 2026 to 24th April 2026.

The project report has not submitted by me for any award of any other degree/diploma of any University/Institute.

Place: Kanpur

Raj Singh

Date:

BCA

Roll No. 23071001273

ACKNOWLEDGEMENT

1. Project plays a vital role in Engineering Education. It gives a student a chance to interact with the practical knowledge of real world. As a part of regular curriculum, I have got a chance to perform my final project. I have started to work on project from Jan 2026 since then many people have made valuable suggestion, which I have incorporated in this work. I take this opportunity to express my gratitude to them.
2. I am beholden to my colleagues who provided me complete knowledge about this project and they were good enough to find time for fruitful discussions.
3. I am also thanking full with my present employer and faculty members of my Institute who gives chance and valuable suggestions.
4. Last but not the least I thank to my family members for their support & encouragement.
5. It will be great pleasure for me if the readers draw useful conclusion from report.

Raj Singh

TABLE OF CONTENTS

Synopsis Report

1.	Project profile		
	1.1	Introduction	
	1.2	Objectives	
	1.3	Project Category	
	1.4	Tools & Platform used	
	1.5	Development Team	
	1.6	System Development Model	
2.	System Analysis		
	2.1	Requirement Analysis	
	2.2	System Requirement Specification	
	•	Hardware Requirement	
	•	Software Requirement	
	2.3	Risk Analysis	
	2.4	Cost Analysis	
	2.5	Project Scheduling	

3.	System Designing		
	3.1	Modules	
	3.2	Database Design	
	3.3	Data Flow Diagram (DFD)	
	3.4	Scope and Limitation	
	3.5	Input Output Specification	
4.	Deployment & Implementation		
	4.1	System Testing	
	4.2	Hosting & Deployment	
	4.3	Security	
	4.4	Maintenance	
5.	Annexure		
	5.1	Bibliography	
	5.2	Softcopy of Project on CD Rom	
6.	Project Images		
7.	Conclusions and future scope		
8.	References		

PROJECT PROFILE

1.1:- INTRODUCTION

The **Bank Management System** is a desktop-based application designed to simplify and automate banking operations. In today's fast-paced digital world, managing financial data manually is time-consuming, error-prone, and inefficient. This project aims to overcome these challenges by providing a secure, reliable, and user-friendly system for handling banking activities efficiently.

This system is developed using **Java (Swing and AWT)** for the graphical user interface and **MySQL** for database management. It enables administrators to manage customer accounts, perform transactions, and maintain records in an organized manner. The application provides essential banking functionalities such as account creation, deposit, withdrawal, balance enquiry, and record management.

The primary objective of this project is to reduce manual paperwork and enhance data accuracy while ensuring secure storage and easy retrieval of customer information. The system is designed with a simple and interactive interface, making it easy for users to operate without requiring advanced technical knowledge. It also ensures better data handling, minimizes human errors, and improves overall operational efficiency.

Moreover, the Bank Management System follows a structured development approach, ensuring that all functionalities are properly designed, implemented, and tested. Although it is a desktop application with limited online capabilities, it effectively demonstrates the integration of programming, database management, and user interface design.

In conclusion, this project serves as a practical implementation of concepts learned in software development and database systems. It provides a strong foundation for building more advanced banking applications in the future with enhanced features such as online access, multi-user support, and real-time transaction processing.

1.2 OBJECTIVES

The main objective of the **Bank Management System** is to develop an efficient, reliable, and user-friendly application that simplifies banking operations and improves data management. The system is designed to replace traditional manual methods with a computerized solution that ensures accuracy, security, and speed in handling customer information.

One of the primary objectives of this project is to reduce manual paperwork and minimize human errors in maintaining banking records. By automating processes such as account creation, deposits, withdrawals, and record management, the system enhances efficiency and saves time for administrators.

Another important objective is to provide secure storage of customer data. The system uses a database to store information safely, ensuring that only authorized users can access and modify the data. This helps in maintaining confidentiality and integrity of sensitive banking information.

The project also aims to offer a user-friendly interface that is easy to understand and operate. Even users with basic computer knowledge can efficiently use the system without facing complexity. This improves usability and ensures smooth interaction with the application.

Additionally, the system focuses on fast and accurate data processing. It allows quick retrieval, updating, and deletion of records, which helps in better decision-making and improved customer service.

Finally, the project aims to build a strong foundation for future enhancements. It can be further upgraded to include advanced features such as online banking, multi-user access, and real-time transaction processing, making it more scalable and efficient.

1.3 PROJECT CATEGORY

The **Bank Management System** is a desktop-based application developed to manage and automate banking operations efficiently. This project falls under the category of a **Database Management System (DBMS)** as it focuses on storing, retrieving, and managing customer data using a structured database. It is also considered a **Management Information System (MIS)** because it helps administrators in decision-making by providing organized and meaningful information.

The application is developed using **Java (Core Java, Swing, and AWT)** for designing the graphical user interface and **MySQL** for backend database management. Being a desktop application, it runs on a local system and does not require an internet connection, making it simple, secure, and easy to use. The project mainly focuses on handling banking activities such as account management and transactions in an efficient and systematic manner.

Key Points of Project Category:

- **Type of Application:** Desktop Application
- **Category:** Database Management System (DBMS)
- **System Type:** Management Information System (MIS)
- **Development Platform:** Java (Swing & AWT)
- **Database Used:** MySQL
- **User Type:** Administrator-based system
- **Nature of Work:** Data storage, retrieval, and transaction processing
- **Environment:** Offline (Standalone system)
- **Purpose:** Automating banking operations and managing customer records efficiently

1.4 TOOLS & PLATFORM USED

The development of the **Bank Management System** involves the use of modern programming tools and platforms to ensure efficiency, reliability, and ease of use. The project is primarily developed using **Java technologies** for application logic and user interface design, along with **MySQL** for database management. These tools provide a strong foundation for building a secure and user-friendly desktop application.

The development environment used is **IntelliJ IDEA Community Edition**, which offers powerful features for coding, debugging, and project management. The graphical user interface (GUI) is designed using **Java Swing and AWT**, enabling smooth interaction between the user and the system. For data storage and handling, **MySQL Server** is used, while **MySQL Workbench** helps in database design and administration. The application runs on a **Windows Operating System**, ensuring compatibility and ease of deployment.

Key Tools & Platforms:

- **Programming Language:** Java (Core Java)
- **GUI Framework:** Java Swing & AWT
- **IDE (Development Tool):** IntelliJ IDEA Community Edition
- **Database:** MySQL Server
- **Database Tool:** MySQL Workbench
- **Operating System:** Windows
- **Platform Type:** Desktop Application (Offline)
- **JDK:** Java Development Kit (for compiling and running the program)

1.5 DEVELOPMENT TEAM

The **Bank Management System** project is developed by a team of three students who worked collaboratively to design, implement, and complete the system successfully. Each team member was assigned a specific role based on their skills and area of interest, which helped in efficient task distribution and timely completion of the project.

The team followed a coordinated approach where responsibilities were divided into **Frontend Development, Backend Development, and Database Management**. This division ensured that each part of the system was developed with focus and accuracy. Regular discussions and teamwork helped in solving problems, improving system design, and integrating all modules smoothly.

Team Roles and Responsibilities:

- **Frontend Developer: (Raj Singh)**
Responsible for designing the graphical user interface using Java Swing and AWT. Focused on creating an interactive, user-friendly, and visually appealing interface for users.
- **Backend Developer: (Raghav Pandey)**
Handled the application logic and functionality of the system. Responsible for implementing core features such as login, transactions, and system operations using Java.
- **Database Developer: (Ragib Irshad)**
Managed the database design and data handling using MySQL. Responsible for creating tables, managing data storage, retrieval, and ensuring data security and integrity.

Key Contributions of the Team:

- Efficient division of work among team members
- Smooth integration of frontend, backend, and database
- Regular communication and problem-solving
- Proper testing and debugging of the system
- Successful completion of the project within the given time

1.6 SYSTEM DEVELOPMENT MODEL

The **Bank Management System** is developed using the **Waterfall Model**, which is a linear and sequential approach to software development. In this model, each phase is completed before moving on to the next phase, ensuring proper planning, clear understanding, and systematic execution of the project. This model is suitable for this project because the requirements are well-defined and do not change frequently.

The Waterfall Model helps in maintaining discipline during development by dividing the project into different stages such as requirement analysis, system design, implementation, testing, and deployment. Each phase has its own importance and contributes to the successful completion of the system. Proper documentation is maintained at every stage, which makes the system easy to understand and manage.

Phases of Waterfall Model:

1. **Requirement Analysis:**

In this phase, all system requirements are identified and analyzed, such as managing customer details, transactions, and security needs.

2. **System Design:**

The overall structure of the system is designed, including modules, database design, and user interface layout.

3. **Implementation (Coding):**

The actual development of the system is done using Java, where all functionalities are coded.

4. **Testing:**

The system is tested to identify and fix errors or bugs, ensuring proper functionality and performance.

5. **Deployment:**

The application is installed on the system and made ready for use by the administrator.

6. **Maintenance:**

After deployment, the system is maintained to fix issues, update features, and improve performance.

SYSTEM ANALYSIS

2.1 REQUIREMENT ANALYSIS

Requirement Analysis is the first and most important phase of the system development process. In this phase, all the needs and expectations of the **Bank Management System** are identified and clearly defined. It helps in understanding what the system should do and how it should perform. Proper requirement analysis ensures that the final system is efficient, user-friendly, and meets all the desired objectives.

The main purpose of this phase is to analyze the problems in the existing manual system and provide a suitable computerized solution. The current manual process of managing customer records and transactions is time-consuming, error-prone, and difficult to maintain. Therefore, the proposed system aims to automate these processes and improve accuracy, speed, and security.

Types of Requirements:

1. Functional Requirements:

These define the basic functions that the system must perform.

- User login and authentication
- Opening a new customer account
- Depositing and withdrawing money
- Balance enquiry and mini statement
- Updating and deleting customer records
- Generating and displaying reports

2. Non-Functional Requirements:

These define the quality and performance aspects of the system.

- **Security:** Only authorized users can access the system
- **Performance:** Fast processing of data and transactions
- **Usability:** Simple and user-friendly interface
- **Reliability:** System should work without failure
- **Maintainability:** Easy to update and modify

2.2 SYSTEM REQUIREMENT SPECIFICATION

The **System Requirement Specification (SRS)** defines the functional and non-functional requirements of the **Bank Management System**. It acts as a blueprint for the development of the system, ensuring that all features and operations are clearly described before implementation. The SRS helps developers understand system behavior, performance expectations, and technical needs, resulting in a well-structured and efficient application.

This system is designed to perform essential banking operations such as account management, transactions, and record handling. It ensures secure access, accurate data processing, and smooth interaction between the user and the system. The requirements are divided into hardware and software components necessary for proper functioning of the application.

1. Functional Requirements:

- Secure login system for authorized users
- Add, update, delete, and search customer records
- Perform banking operations like deposit and withdrawal
- Display account details, balance enquiry, and mini statements
- Generate reports and maintain transaction history

2. Non-Functional Requirements:

- **Security:** Protection of sensitive customer data
- **Performance:** Quick response and fast data processing
- **Usability:** Easy-to-use graphical interface
- **Reliability:** System should work smoothly without failure
- **Scalability:** Can be upgraded with new features in future

3. Hardware Requirements:

- Computer/Laptop
- Intel Core i3 or equivalent processor
- Minimum 4 GB RAM
- 250 GB Hard Disk (or above)
- Keyboard and Mouse

- Monitor
- Stable Power Supply

4. Software Requirements:

- Java Development Kit (JDK)
- IntelliJ IDEA Community Edition (IDE)
- MySQL Server
- MySQL Workbench
- Windows Operating System

Conclusion:

The System Requirement Specification ensures that all technical and functional aspects of the project are clearly defined. It helps in developing a reliable, secure, and efficient Bank Management System that meets user needs and performs effectively.

2.3 RISK ANALYSIS

Risk Analysis is an important phase in system development that involves identifying potential problems that may affect the performance, security, or success of the **Bank Management System**. It helps in understanding possible risks in advance and taking preventive measures to minimize their impact. Proper risk management ensures that the system remains reliable, secure, and efficient during its operation.

In this project, risks may arise due to technical issues, human errors, or system failures. Since the system handles sensitive customer data, security risks are also a major concern. Therefore, appropriate strategies are implemented to reduce these risks and ensure smooth functioning of the application.

Types of Risks and Mitigation:

1. Technical Risks:

- Software bugs or errors in coding
- System crash or application failure
- Compatibility issues with system environment

Mitigation: Proper testing, debugging, and use of reliable tools.

2. Data Security Risks:

- Unauthorized access to customer data
- Data leakage or misuse

Mitigation: Secure login system, restricted access, and data validation.

3. Data Loss Risks:

- Loss of data due to system failure or power issues
- Accidental deletion of records

Mitigation: Regular data backup and recovery mechanisms.

4. Human Errors:

- Incorrect data entry by the user
- Lack of user knowledge

Mitigation: Input validation and user-friendly interface.

5. Performance Risks:

- Slow system performance with large data
- Delay in processing transactions

Mitigation: Efficient coding and optimized database design.

2.4 COST ANALYSIS

Cost Analysis is an important part of system development that helps in estimating the total expenses involved in developing and implementing the **Bank Management System**. It includes both direct and indirect costs such as hardware, software, development, and maintenance. The main objective of cost analysis is to ensure that the project is economically feasible and can be developed within a reasonable budget.

In this project, the overall cost is relatively low because most of the tools and technologies used are free or open-source. The development is carried out by students, so there is no labor cost involved. The primary expenses are related to basic hardware and electricity consumption, making the project highly cost-effective.

Types of Costs:

1. Hardware Cost:

- Computer or Laptop
- Input/Output devices (Keyboard, Mouse, Monitor)
- Power supply and electricity

2. Software Cost:

- Java Development Kit (JDK) – Free
- IntelliJ IDEA Community Edition – Free
- MySQL Server & Workbench – Free

3. Development Cost:

- No major cost as the project is developed by students
- Time and effort invested in coding and testing

4. Maintenance Cost:

- Cost for system updates and improvements
- Data backup and minor troubleshooting

2.5 PROJECT SCHEDULING

Project Scheduling is the process of planning and organizing different phases of the **Bank Management System** to ensure timely and successful completion. It involves dividing the entire project into smaller tasks and assigning a specific time period to each phase. Proper scheduling helps in managing time efficiently, avoiding delays, and ensuring smooth progress of the project.

In this project, the development process is carried out in a structured manner, starting from requirement analysis to final implementation. Each phase is completed step by step, ensuring that all tasks are properly planned and executed. This systematic approach helps in maintaining quality and achieving project objectives within the given timeframe.

Project Development Phases & Timeline:

- **Requirement Analysis (1 Week):**
Understanding system needs and defining project objectives.
- **System Design (1 Week):**
Designing system architecture, modules, and database structure.
- **Implementation / Coding (2–3 Weeks):**
Developing the application using Java and integrating with MySQL.
- **Testing (1 Week):**
Identifying and fixing bugs, ensuring proper functionality.
- **Deployment (2–3 Days):**
Installing and setting up the system on the user's computer.
- **Documentation (1 Week):**
Preparing project report and related documents.

Benefits of Project Scheduling:

- Ensures timely completion of the project
- Helps in proper task management

SYSTEM DESIGNING

3.1 MODULES

The **Bank Management System** is divided into different modules to ensure smooth, organized, and efficient functioning of the application. Each module is responsible for performing a specific task, which makes the system easy to develop, understand, and maintain. Modular design helps in reducing complexity and allows independent development and testing of each part of the system.

These modules work together to handle all banking operations such as account management, transactions, and user authentication. Proper integration of all modules ensures that the system performs accurately and efficiently.

Main Modules of the System:

1. Login Module:

Provides secure access to the system using valid credentials (Card Number and PIN). It ensures that only authorized users can access the application.

2. Open Account (Signup) Module:

Allows the creation of a new customer account by collecting personal and account-related details through a form.

3. Deposit Module:

Enables users to deposit money into their account and updates the balance in the database.

4. Withdrawal Module:

Allows users to withdraw money from their account after checking for sufficient balance.

5. Balance Enquiry Module:

Displays the current balance of the customer's account instantly.

6. PIN Change Module:

Allows users to change or update their PIN for security purposes.

7. Fast Withdrawal Module:

Provides quick withdrawal options with predefined amounts for faster transactions.

8. Mini Statement Module:

Shows a summary of recent transactions made by the user.

9. Report/Print Module:

Generates and prints customer details and transaction records.

10. Exit Module:

Allows the user to safely close the application.

3.2 DATABASE DESIGN

The **Database Design** of the Bank Management System is an essential part of the project, as it defines how data is stored, organized, and managed efficiently. The system uses **MySQL** as the database to handle large amounts of customer and transaction data in a structured manner. A well-designed database ensures data integrity, reduces redundancy, and allows fast retrieval of information.

The database is designed using tables that store different types of information such as customer details, account information, login credentials, and transaction records. Each table is connected using keys (Primary Key and Foreign Key), which helps in maintaining relationships between different data entities. This structured approach ensures smooth functioning of the system and accurate data handling.

Main Tables in Database:

1. Customer / Signup Table:

Stores personal details of customers such as name, address, date of birth, and contact information.

2. Account Details Table (Signup2/Signup3):

Contains additional details like account type, income, education, and other related information.

3. Login Table:

Stores login credentials such as Card Number and PIN for authentication.

4. Transaction Table (Deposit/Withdrawal):

Maintains records of all transactions including date, type (deposit/withdrawal), and amount.

Key Features of Database Design:

- **Primary Key:** Unique identification for each record (e.g., Account Number/Card Number)
- **Data Integrity:** Ensures accuracy and consistency of data

3.3 DATA FLOW DIAGRAM (DFD)

The **Data Flow Diagram (DFD)** is a graphical representation that shows how data flows through the **Bank Management System**. It illustrates the interaction between different components such as users (administrator), processes, and the database. The DFD helps in understanding how input data is processed and converted into meaningful output within the system.

In this system, the administrator provides input such as customer details, login credentials, and transaction requests. This data is processed by various modules like login verification, account management, deposit, and withdrawal. After processing, the data is stored in the database and can be retrieved whenever required. The DFD ensures clarity in system design and helps in identifying the flow of information step by step.

Main Components of DFD:

- **External Entity:** Administrator/User
- **Processes:** Login, Account Management, Transaction Processing
- **Data Store:** Database (MySQL)
- **Data Flow:** Movement of data between user, processes, and database

Levels of DFD:

1. Level 0 (Context Diagram):

Shows the overall system as a single process. The user interacts with the system by providing input and receiving output.

2. Level 1 DFD:

Breaks down the system into multiple processes such as login, account creation, deposit, withdrawal, and report generation.

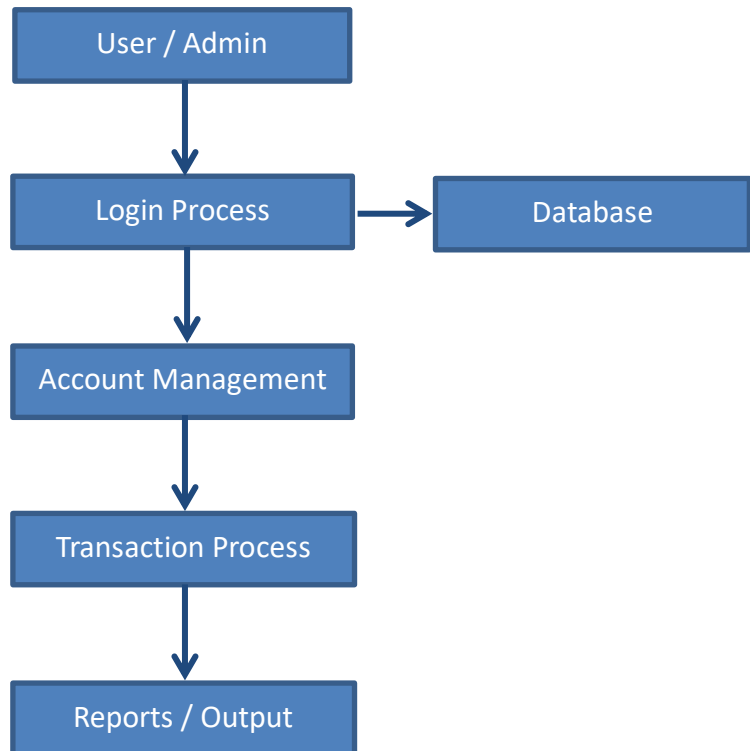
Basic Flow of Data:

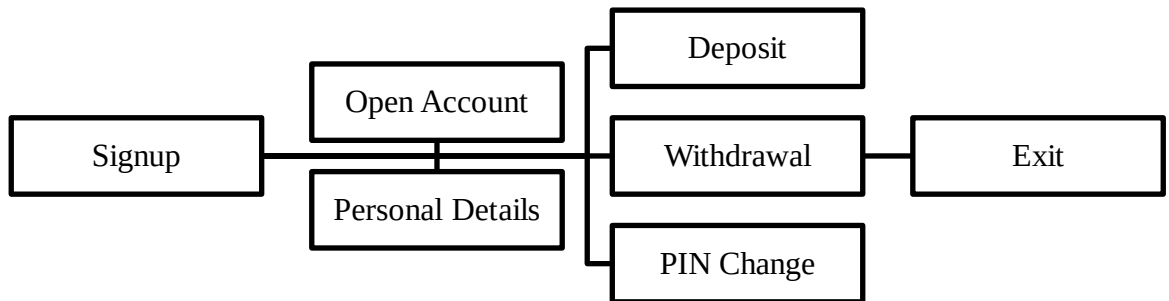
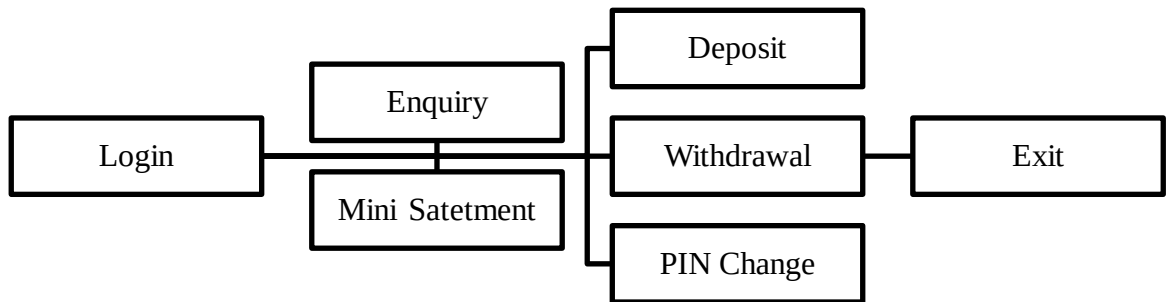
- User enters login details → System verifies → Access granted
- User inputs customer/account data → System processes → Data stored in database
- User performs transactions → System updates records → Output displayed

- System retrieves data → Displays reports and account details

Diagram Idea (for drawing in exam):

You can draw simple DFD like this:





3.4 SCOPE AND LIMITATION

The **Scope** of the **Bank Management System** defines the extent and capabilities of the system, while **Limitations** highlight the constraints under which the system operates. This project is designed to automate basic banking operations and improve the efficiency of managing customer data in a secure and organized manner.

The system provides functionalities such as account creation, deposit, withdrawal, balance enquiry, and record management. It helps in reducing manual work, minimizing errors, and improving the speed of operations. The application is developed as a desktop-based system, mainly for administrative use, ensuring easy access and control over customer data.

Scope of the System:

- Automates banking operations and reduces manual paperwork
- Maintains customer records in a structured database
- Provides secure login for authorized access
- Supports transactions like deposit and withdrawal
- Allows quick data retrieval, updating, and deletion
- Generates reports and displays account information
- Improves accuracy, efficiency, and reliability of data handling

Limitations of the System:

- Works only as a **desktop application** (no online access)
- Limited to **single-user (administrator)** environment
- Does not support advanced features like online banking or UPI
- Requires manual data entry, which may lead to errors
- Dependent on system hardware and local database setup
- Limited scalability for large-scale banking operations

3.5 INPUT OUTPUT SPECIFICATION

The **Input Output Specification** defines the type of data entered into the **Bank Management System** and the results produced by the system after processing. It ensures that the system handles data accurately and provides meaningful output to the user. Proper specification of input and output helps in maintaining efficiency, reliability, and correctness of the system.

In this system, inputs are provided by the administrator through a graphical user interface, where details such as customer information, account data, and transaction requests are entered. The system processes this data and generates appropriate outputs like account details, transaction results, and reports.

Input Specification:

The input to the system includes various types of data entered by the user:

- **Login Details:** Card Number and PIN for authentication
- **Customer Information:** Name, address, date of birth, contact details
- **Account Details:** Account type, initial balance, and other relevant data
- **Transaction Data:** Deposit and withdrawal amounts
- **Search Data:** Account number or customer name for retrieving records

Output Specification:

The output generated by the system after processing the input includes:

- **Login Status:** Successful login or error message
- **Customer Details:** Display of stored customer information
- **Transaction Results:** Confirmation of deposit or withdrawal
- **Balance Enquiry:** Current account balance
- **Mini Statement:** Summary of recent transactions
- **Reports:** Printed or displayed records of customer data

DEPLOYMENT & IMPLEMENTATION

4.1 SYSTEM TESTING

System Testing is an important phase in the development of the **Bank Management System**, where the complete system is tested to ensure that all functionalities are working correctly. The main purpose of testing is to identify errors, bugs, or defects and fix them before the system is deployed. It ensures that the application performs as expected and meets all user requirements.

In this project, testing is performed after the implementation phase to verify each module such as login, account creation, deposit, withdrawal, and data management. The system is tested under different conditions to check its reliability, performance, and accuracy. Proper testing helps in improving the quality of the system and ensures a smooth user experience.

Types of Testing Performed:

1. Unit Testing:

Each module (Login, Deposit, Withdrawal, etc.) is tested individually to ensure correct functionality.

2. Integration Testing:

All modules are combined and tested together to verify proper interaction between them.

3. System Testing:

The complete system is tested as a whole to ensure it meets all requirements.

4. User Acceptance Testing (UAT):

The system is tested by users to confirm that it works according to their expectations.

Test Cases Examples:

- Checking login with valid and invalid credentials
- Verifying deposit and withdrawal operations
- Testing data entry and record updates
- Checking database connectivity and data retrieval
- Validating error messages and system responses

Benefits of System Testing:

- Detects and removes errors or bugs
- Ensures system reliability and performance
- Improves user satisfaction
- Validates system requirements

4.2 HOSTING & DEPLOYMENT

Hosting and Deployment refer to the process of making the **Bank Management System** ready for actual use by installing and configuring it on a system. Since this project is a desktop-based application, it does not require web hosting or internet servers. Instead, it is deployed locally on a computer where the administrator can access and use the system efficiently.

The deployment process involves setting up the required software such as Java and MySQL, configuring the database, and running the application. Proper deployment ensures that the system works smoothly without errors and all functionalities are accessible to the user.

Deployment Process:

- Install **Java Development Kit (JDK)** on the system
- Install and configure **MySQL Server**
- Create the required database and tables
- Import database data (if available)
- Compile and run the Java application
- Test the system after installation

Hosting Type:

- **Local Hosting (Offline):**

The system is hosted on a local machine and does not require internet connectivity.

Requirements for Deployment:

- Computer or Laptop with required specifications
- Installed Java and MySQL environment
- Proper database configuration
- Application files (.java / .jar)

Advantages of Local Deployment:

- No internet dependency
- Faster performance
- Better data control and security

4.3 SECURITY

Security is a crucial aspect of the **Bank Management System** as it deals with sensitive customer and financial data. The system is designed to ensure that only authorized users can access and manage the data. Proper security measures are implemented to protect the system from unauthorized access, data leakage, and misuse.

The application uses a secure login mechanism where users must enter valid credentials such as Card Number and PIN. Without proper authentication, access to the system is denied. This ensures that confidential data remains protected at all times.

Security Measures:

- **User Authentication:** Only authorized users can log in using valid credentials
- **Data Validation:** Input fields are validated to prevent incorrect or harmful data
- **Access Control:** Restricted access to sensitive operations
- **Data Protection:** Customer data is securely stored in the database
- **Error Handling:** Proper error messages to avoid system crashes

4.4 MAINTENANCE

Maintenance is the process of updating and improving the system after deployment. It ensures that the **Bank Management System** continues to function efficiently and remains free from errors over time. Regular maintenance helps in improving performance, fixing bugs, and adding new features if required.

Maintenance is important because software systems may face issues such as performance degradation, data errors, or changes in user requirements. Proper maintenance ensures long-term usability and reliability of the system.

Types of Maintenance:

- **Corrective Maintenance:** Fixing bugs and errors in the system
- **Adaptive Maintenance:** Updating the system according to new requirements
- **Perfective Maintenance:** Improving system performance and features
- **Preventive Maintenance:** Taking measures to avoid future issues

ANNEXURE

5.1 BIBLIOGRAPHY

1. GeeksforGeeks “Flask Web Development Tutorials.” Available at:
<https://www.geeksforgeeks.org> (Accessed 2025).
2. Incredible India “Official Tourism Website for Destination Data and Historical Facts.” Available at:
<https://www.incredibleindia.gov.in/en> (Accessed 2025).
3. Tutorials Point “Web Development and Database Tutorials.” Available at:
<https://www.tutorialspoint.com> (Accessed 2025).
4. W3Schools “Web Development Tutorials (HTML, CSS, JavaScript).” Available at:
<https://www.w3schools.com> (Accessed 2025).
5. YouTube “Flask Web Development and Python Tutorials.” Available at:
<https://www.youtube.com> (Accessed 2025).

5.2 SOFTCOPY OF PROJECT ON CD-ROM

The softcopy of the **Bank Management System** project is provided on a CD-ROM for easy access, storage, and submission. It contains all the necessary files related to the project, including source code, database, and documentation. This allows users, evaluators, or faculty members to review, execute, and understand the project conveniently.

The CD-ROM includes the complete project folder, which is properly organized to ensure easy navigation. All required files are arranged systematically so that the project can be executed without any confusion.

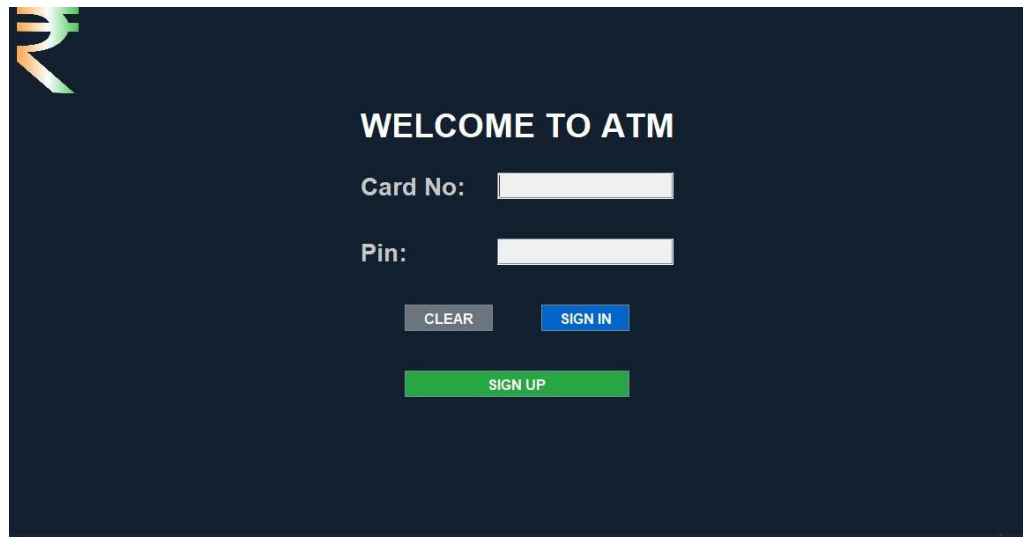
Contents of CD-ROM:

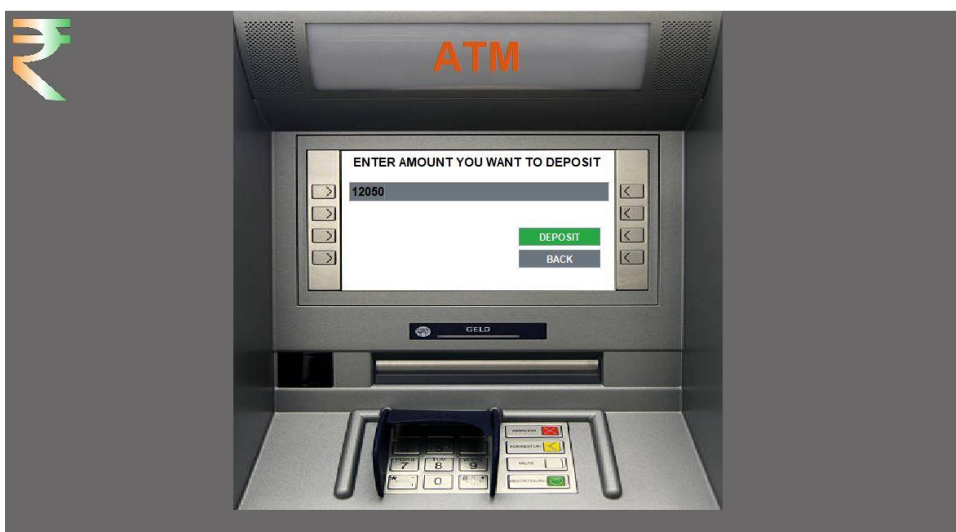
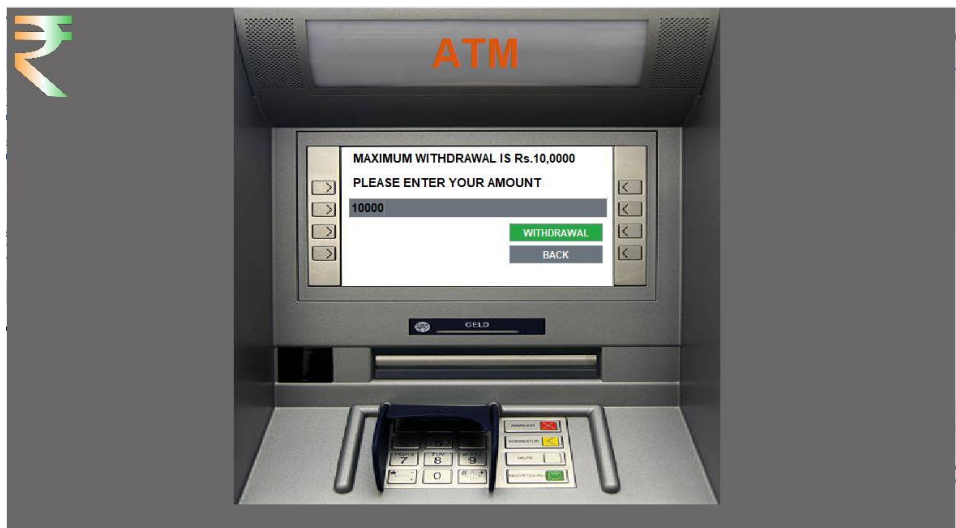
- **Source Code:** All Java files (.java) used in the project
- **Executable File:** Compiled file (.jar) to run the application
- **Database Files:** MySQL database scripts (.sql)
- **Project Report:** Documentation in PDF or Word format
- **Images/Screenshots:** System interface and output screens
- **README File:** Instructions to run the project

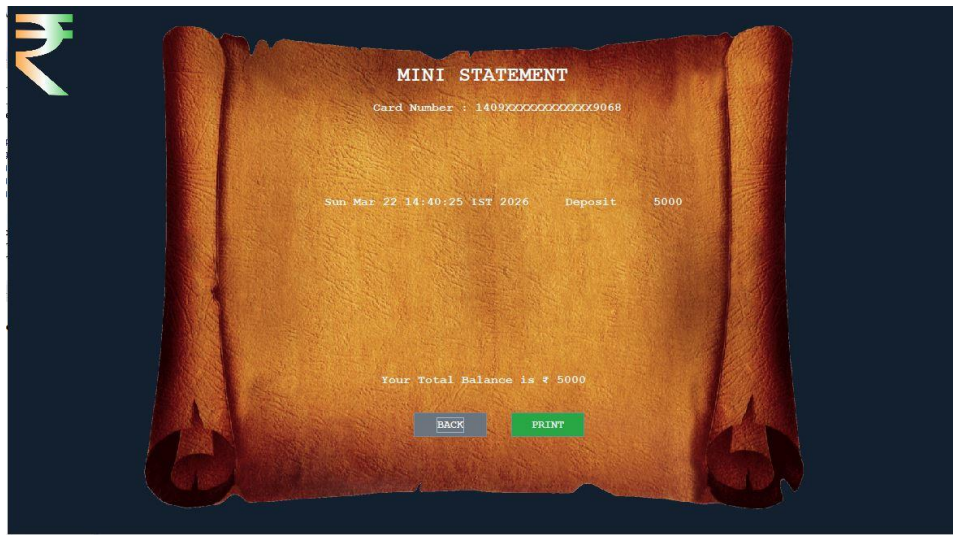
How to Use the CD-ROM:

- Insert the CD into the computer
- Open the project folder
- Install required software (Java & MySQL if not installed)
- Import the database using the .sql file
- Run the application using the .jar file or source code

PROJECT IMAGES







The screenshot shows an 'APPLICATION FORM NO. 8894' on a dark blue background with a stylized Indian Rupee symbol (₹) in the top left corner. The form is titled 'PAGE 1' and 'PERSONAL DETAILS'. It contains the following fields and options:

Name		Father's Name	
Gender	☐ Male ☐ Female ☐ Other	Date of Birth	
Marital Status	☒ Married ☐ Unmarried	Email Address	
Address		City	
Pin Code		State	

At the bottom, there are 'BACK' and 'NEXT' buttons.



APPLICATION FORM NO. 8894
PAGE 2
ADDITIONAL DETAILS

Religion	Hindu	Category	General
Education	Non-Graduate	Income	Null
Occupation	Salaried	Pan No	
Aadhar No		Senior Citizen	☐ Yes ☐ No
Exiting A/C	☐ Yes ☐ No		



APPLICATION FORM NO. 8894
PAGE 3
ACCOUNT DETAILS

Account Type
☒ Saving Account
☐ Fixed Deposit Account
☐ Current Account
☐ Recurring Deposit Account

Card Number
(Your 16 Digit Card Number)
XXXX-XXXX-XXXX-4841
(It would appear on ATM card/cheque book and statement)

Service Required
☐ ATM Card ☐ Internet Banking ☐ Mobile Banking
☐ Cheque Book ☐ E-Statement ☐ Email Alert

Pin
XXXX
(4-Digit Password)

☒ I hereby declare that the above entered details are correct to the best of my knowledge.

CONCLUSIONS AND FUTURE SCOPE

CONCLUSION

The **Bank Management System** is a desktop-based application developed to simplify and automate banking operations. The project successfully achieves its main objective of replacing manual processes with a computerized system that is efficient, reliable, and user-friendly. It provides essential functionalities such as account creation, deposit, withdrawal, balance enquiry, and record management.

Throughout the development of this project, various concepts of software engineering, database management, and programming have been applied. The use of Java for application development and MySQL for database management ensures smooth functioning and secure handling of data. The system is designed in a modular way, making it easy to understand, maintain, and extend in the future.

The project also demonstrates the importance of proper planning, requirement analysis, system design, and testing. Each phase of development plays a significant role in ensuring the quality and performance of the final system. Testing ensures that the system is free from major errors and performs all operations accurately.

In addition, the system provides a user-friendly interface that allows administrators to perform operations easily without requiring advanced technical knowledge. It reduces manual effort, minimizes errors, and improves the speed and accuracy of data processing. The use of a structured database ensures secure storage and quick retrieval of information.

Although the system has some limitations, such as being a desktop-based and single-user application, it serves as a strong foundation for understanding real-world banking systems. The project highlights the practical implementation of theoretical concepts learned during the course.

In conclusion, the **Bank Management System** is a successful project that meets its objectives and provides an efficient solution for managing banking operations. It can be further enhanced with advanced features to make it more scalable and suitable for real-world applications.

FUTURE SCOPE

The current system provides basic banking functionalities, but it can be further improved by adding advanced features to enhance usability, security, and performance. Future enhancements will make the system more powerful and suitable for real-world banking environments.

Possible Future Enhancements:

- **Online Banking System:** Enable internet-based access for users
- **Multi-User Support:** Allow multiple users to access the system simultaneously
- **Mobile Application Integration:** Develop a mobile app for easy access
- **UPI & Digital Payment Integration:** Support modern payment methods
- **Advanced Security:** Implement OTP verification and encryption
- **Cloud Database:** Store data on cloud for better accessibility
- **ATM Simulation Features:** Add ATM-like interface and operations
- **Report Generation in PDF:** Export reports in PDF format
- **Improved UI/UX:** More attractive and interactive interface

REFERENCES

1. Oracle. Java Platform Standard Edition Documentation, Oracle Inc., 2025.
2. MySQL AB. MySQL 8.0 Reference Manual. Oracle Corporation.
3. GeeksforGeeks. Java Swing & JDBC Tutorials.
4. W3Schools. SQL and Database Connectivity Reference.
5. TutorialsPoint. Object-Oriented Programming in Java.
6. JetBrains. IntelliJ IDEA Official Documentation.
7. YouTube Channel – Java Programming & GUI Projects (Core Java 2024).

CODING

1.Login.java (class)

```
package bank.management.system;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.ResultSet;

public class Login extends JFrame implements ActionListener {

    JLabel label1, label2, label3;

    JTextField textField2;

    JPasswordField passwordField3;

    JButton button1, button2, button3;

    Login() {
        super("CUSTOMER SIGNIN/SIGNUP");

        getContentPane().setBackground(new Color(18, 32, 47)); // Dark Navy Background

        ImageIcon i1 = new ImageIcon(ClassLoader.getResource("icons/rupes.png"));
        Image i2 = i1.getImage().getScaledInstance(100, 100, Image.SCALE_DEFAULT);
        ImageIcon i3 = new ImageIcon(i2); JLabel image = new JLabel(i3);

        image.setBounds(10, 10, 70, 100);

        add(image);

        // LABEL

        label1 = new JLabel("WELCOME TO ATM");

        label1.setForeground(new Color(255, 255, 255)); // White

        label1.setFont(new Font("AvantGarde", Font.BOLD, 38));

        label1.setBounds(400, 125, 500, 40);

        add(label1);

        label2 = new JLabel("Card No:");
```

```
label2.setFont(new Font("AvantGarde", Font.BOLD, 28));
label2.setForeground(new Color(200, 200, 200)); // Light Gray
label2.setBounds(400,200,375,30);
add(label2);

textField2 = new JTextField(15);
textField2.setBounds(555,200,200,30);
textField2.setFont(new Font("AvantGarde", Font.BOLD,18));
textField2.setBackground(new Color(240,240,240)); // Soft white
textField2.setForeground(Color.BLACK);
add(textField2);

label3 = new JLabel("Pin: ");
label3.setFont(new Font("AvantGarde", Font.BOLD, 28));
label3.setForeground(new Color(200, 200, 200)); // Light Gray
label3.setBounds(400,275,375,30);
add(label3);

passwordField3 = new JPasswordField(15);
passwordField3.setBounds(555,275,200,30);
passwordField3.setFont(new Font("AvantGarde", Font.BOLD, 28));
passwordField3.setBackground(new Color(240,240,240));
passwordField3.setForeground(Color.BLACK);
add(passwordField3);

button1 = new JButton("SIGN IN");
button1.setFont(new Font("AvantGarde", Font.BOLD, 16));
button1.setForeground(Color.WHITE);
button1.setBackground(new Color(0, 102, 204)); // Professional Blue
button1.setBounds(605,350,100, 30);
button1.addActionListener(this);
add(button1);

button2 = new JButton("CLEAR");
```

```
button2.setFont(new Font("AvantGarde", Font.BOLD, 16));
button2.setForeground(Color.WHITE);
button2.setBackground(new Color(108, 117, 125)); // Soft Gray
button2.setBounds(450,350,100, 30);
button2.addActionListener(this);
add(button2);

button3 = new JButton("SIGN UP");
button3.setFont(new Font("AvantGarde", Font.BOLD, 16));
button3.setForeground(Color.WHITE);
button3.setBackground(new Color(40, 167, 69)); // Modern Green
button3.setBounds(450,425,255, 30);
button3.addActionListener(this);
add(button3);

setSize(1170,650);
setLocation(100,50);
setLayout(null);
// setUndecorated(true);
setVisible(true);
}

@Override
public void actionPerformed(ActionEvent e)
{
    try{
        if (e.getSource()==button1){
            Conn c = new Conn();

            String CardNo = textField2.getText();

            String Pin = passwordField3.getText();

            String Q = "select * from Login where CardNo = '"+CardNo+"' and Pin = '"+Pin+"'";

            ResultSet resultSet = c.statement.executeQuery(Q);
```

```
        if (resultSet.next()){
            setVisible(false);
            new Mainclasss(Pin);
        }else {
            UIManager.put("OptionPane.background",new Color(30,144,255) );
            UIManager.put("Panel.background",new Color(30,144,225));
            JOptionPane.showMessageDialog(null,"The Card Number or PIN you entered is
incorrect. \nPlease check and try again.", "Authentication
Failed",JOptionPane.ERROR_MESSAGE);
        }
    }
    else if (e.getSource()==button2) {
        textField2.setText("");
        passwordField3.setText("");
    }
    else if(e.getSource()==button3){
        new Signup();
        setVisible(false);
    }
}
catch (Exception E){
    E.printStackTrace();
}
}
public static void main(String[] args) {
    new Login();
}
}
```

2.Signup.java (class)

```
package bank.management.system;

import com.toedter.calendar.JDateChooser;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;

public class Signup extends JFrame implements ActionListener {

    JRadioButton r1, r2, r3, m1, m2;

    JButton next,back;

    JTextField textName, textFname, textEmail, textAdd, textcity, textState, textPin;

    JDateChooser dateChooser;

    Random ran = new Random();

    long first4 = (ran.nextLong() % 9000L) + 1000L;

    String first = " " + Math.abs(first4);

    Signup() {

        super("PAGE-1 APPLICATION FORM");

        getContentPane().setBackground(new Color(18, 32, 47)); // Dark Navy Background

        ImageIcon i1 = new ImageIcon(ClassLoader.getResource("icons/rupes.png"));

        Image i2 = i1.getImage().getScaledInstance(100, 100, Image.SCALE_DEFAULT);

        ImageIcon i3 = new ImageIcon(i2); JLabel image = new JLabel(i3);

        image.setBounds(10, 10, 70, 100);

        add(image);

        JLabel label1 = new JLabel("APPLICATION FORM NO." + first);

        label1.setForeground(Color.WHITE);

        label1.setFont(new Font("AvantGarde", Font.BOLD, 38));

        label1.setBounds(300, 35, 600, 40);

        add(label1);
```

```
JLabel label2 = new JLabel("PAGE 1");
label2.setForeground(new Color(200,200,200));
label2.setBounds(535, 75, 100, 30);
label2.setFont(new Font("AvantGarde", Font.BOLD, 22));
add(label2);

JLabel label3 = new JLabel("PERSONAL DETAILS");
label3.setForeground(new Color(0, 123, 255)); // Highlight Blue
label3.setBounds(460, 105, 250, 30);
label3.setFont(new Font("AvantGarde", Font.BOLD, 22));
add(label3);

JLabel labelName = new JLabel("Name");
labelName.setForeground(new Color(220,220,220));
labelName.setBounds(100, 190, 100, 30);
labelName.setFont(new Font("AvantGarde", Font.BOLD, 22));
add(labelName);

textFieldName = new JTextField();
textFieldName.setBackground(new Color(240,240,240));
textFieldName.setForeground(Color.BLACK);
textFieldName.setBounds(200, 190, 275, 30);
textFieldName.setFont(new Font("AvantGarde", Font.BOLD, 14));
add(textFieldName);

JLabel labelFName = new JLabel("Father's Name");
labelFName.setForeground(new Color(220,220,220));
labelFName.setFont(new Font("AvantGarde", Font.BOLD, 20));
labelFName.setBounds(565, 190, 200, 30);
add(labelFName);

textFieldName = new JTextField();
textFieldName.setBackground(new Color(240,240,240));
textFieldName.setForeground(Color.BLACK);
```

```
textFname.setFont(new Font("AvantGarde", Font.BOLD, 14));
textFname.setBounds(765, 190, 275, 30);
add(textFname);

JLabel DOB = new JLabel("Date of Birth");
DOB.setForeground(new Color(220,220,220));
DOB.setFont(new Font("AvantGarde", Font.BOLD, 20));
DOB.setBounds(565, 240, 200, 30);
add(DOB);

dateChooser = new JDateChooser();
dateChooser.setBackground(new Color(240,240,240));
dateChooser.setBounds(765, 240, 275, 30);
add(dateChooser);

JLabel labelG = new JLabel("Gender");
labelG.setForeground(new Color(220,220,220));
labelG.setFont(new Font("AvantGarde", Font.BOLD, 20));
labelG.setBounds(100, 240, 200, 30);
add(labelG);

r1 = new JRadioButton("Male");
r1.setBackground(new Color(18, 32, 47));
r1.setForeground(Color.WHITE);
r1.setFont(new Font("AvantGarde", Font.BOLD, 14));
r1.setBounds(200, 240, 70, 30);
add(r1);

r2 = new JRadioButton("Female");
r2.setBackground(new Color(18, 32, 47));
r2.setForeground(Color.WHITE);
r2.setFont(new Font("AvantGarde", Font.BOLD, 14));
r2.setBounds(282, 240, 90, 30);
add(r2);
```

```
r3 = new JRadioButton("Other");
r3.setBackground(new Color(18, 32, 47));
r3.setForeground(Color.WHITE);
r3.setFont(new Font("AvantGarde", Font.BOLD, 14));
r3.setBounds(382, 240, 90, 30);
add(r3);

ButtonGroup buttonGroup = new ButtonGroup();
buttonGroup.add(r1);
buttonGroup.add(r2);
buttonGroup.add(r3);

JLabel labelMs = new JLabel("Marital Status");
labelMs.setForeground(new Color(220,220,220));
labelMs.setFont(new Font("AvantGarde", Font.BOLD, 20));
labelMs.setBounds(100, 290, 200, 30);
add(labelMs);

m1 = new JRadioButton("Married");
m1.setBackground(new Color(18, 32, 47));
m1.setForeground(Color.WHITE);
m1.setFont(new Font("AvantGarde", Font.BOLD, 14));
m1.setBounds(250, 290, 100, 30);
add(m1);

m2 = new JRadioButton("Unmarried");
m2.setBackground(new Color(18, 32, 47));
m2.setForeground(Color.WHITE);
m2.setFont(new Font("AvantGarde", Font.BOLD, 14));
m2.setBounds(375, 290, 100, 30);
add(m2);

ButtonGroup buttonGroup1 = new ButtonGroup();
buttonGroup1.add(m1);
```

```
buttonGroup1.add(m2);

JLabel labelEmail = new JLabel("Email Address");

labelEmail.setForeground(new Color(220,220,220));

labelEmail.setFont(new Font("AvantGarde", Font.BOLD, 20));

labelEmail.setBounds(565, 290, 200, 30);

add(labelEmail);

textEmail = new JTextField();

textEmail.setBackground(new Color(240,240,240));

textEmail.setForeground(Color.BLACK);

textEmail.setFont(new Font("AvantGarde", Font.BOLD, 14));

textEmail.setBounds(765, 290, 275, 30);

add(textEmail);

JLabel labelAdd = new JLabel("Address");

labelAdd.setForeground(new Color(220,220,220));

labelAdd.setFont(new Font("AvantGarde", Font.BOLD, 20));

labelAdd.setBounds(100, 340, 200, 30);

add(labelAdd);

textAdd = new JTextField();

textAdd.setBackground(new Color(240,240,240));

textAdd.setForeground(Color.BLACK);

textAdd.setFont(new Font("AvantGarde", Font.BOLD, 14));

textAdd.setBounds(200, 340, 275, 30);

add(textAdd);

JLabel labelCity = new JLabel("City");

labelCity.setForeground(new Color(220,220,220));

labelCity.setFont(new Font("AvantGarde", Font.BOLD, 20));

labelCity.setBounds(565, 340, 200, 30);

add(labelCity);

textcity = new JTextField();
```

```
textcity.setBackground(new Color(240,240,240));
textcity.setForeground(Color.BLACK);
textcity.setFont(new Font("AvantGarde", Font.BOLD, 14));
textcity.setBounds(765, 340, 275, 30);
add(textcity);

JLabel labelPin = new JLabel("Pin Code");
labelPin.setForeground(new Color(220,220,220));
labelPin.setFont(new Font("AvantGarde", Font.BOLD, 20));
labelPin.setBounds(100, 390, 200, 30);
add(labelPin);

textPin = new JTextField();
textPin.setBackground(new Color(240,240,240));
textPin.setForeground(Color.BLACK);
textPin.setFont(new Font("AvantGarde", Font.BOLD, 14));
textPin.setBounds(200, 390, 275, 30);
add(textPin);

JLabel labelstate = new JLabel("State");
labelstate.setForeground(new Color(220,220,220));
labelstate.setFont(new Font("AvantGarde", Font.BOLD, 20));
labelstate.setBounds(565, 390, 200, 30);
add(labelstate);

textState = new JTextField();
textState.setBackground(new Color(240,240,240));
textState.setForeground(Color.BLACK);
textState.setFont(new Font("AvantGarde", Font.BOLD, 14));
textState.setBounds(765, 390, 275, 30);
add(textState);

back = new JButton("BACK");
back.setFont(new Font("AvantGarde", Font.BOLD, 14));
```

```
back.setForeground(Color.WHITE);

back.setBackground(new Color(108, 117, 125)); // Soft Gray

back.setBounds(460, 500, 90, 30);

back.addActionListener(this);

add(back);


next = new JButton("NEXT");

next.setFont(new Font("AvantGarde", Font.BOLD, 14));

next.setForeground(Color.WHITE);

next.setBackground(new Color(0, 123, 255)); // Professional Blue

next.setBounds(600, 500, 90, 30);

next.addActionListener(this);

add(next);

setLayout(null);

setSize(1170, 650);

setLocation(100, 50);

//setUndecorated(true);

setVisible(true);

}

@Override

public void actionPerformed(ActionEvent e) {

    String FormNo = first;

    String Name = textName.getText();

    String FatherName = textFname.getText();

    String Gender = null;

    if (r1.isSelected()) {

        Gender = "Male";

    } else if (r2.isSelected()) {

        Gender = "Female";

    }

}
```

```
    } else if (r3.isSelected()) {  
        Gender = "Other";  
    }  
  
    String DOB = ((JTextField) dateChooser.getDateEditor().getUiComponent()).getText();  
    String MartialStatus = null;  
    if (m1.isSelected()) {  
        MartialStatus = "Married";  
    } else if (m2.isSelected()) {  
        MartialStatus = "Unmarried";  
    }  
  
    String Email = textEmail.getText();  
    String Address = textAdd.getText();  
    String City = textcity.getText();  
    String PinCode = textPin.getText();  
    String State = textState.getText();  
    try {  
        if (e.getSource()==back) {  
            new Login();  
            setVisible(false);  
        } else if (textName.getText().equals("")) {  
            UIManager.put("OptionPane.background",new Color(30,144,255) );  
            UIManager.put("Panel.background",new Color(30,144,225));  
            JOptionPane.showMessageDialog(null, "All fields are mandatory. Please ensure no  
field is left blank.", "Validation Error",JOptionPane.INFORMATION_MESSAGE);  
        } else if (e.getSource()==next){  
            Conn con1 = new Conn();  
            String Q = "insert into Signup values('"+ FormNo + "','"+ Name + "','"+  
FatherName + "','"+ Gender + "','"+ DOB + "','"+ MartialStatus + "','"+ Email + "','"+  
Address + "','"+ City + "','"+ PinCode + "','"+ State + "')";  
            con1.statement.executeUpdate(Q);
```

```
        new Signup2(FormNo);
        setVisible(false);
    }
} catch (Exception E) {
    E.printStackTrace();
}
}
public static void main (String[] args)
{
    new Signup();
}
}
```

3.Signup2.java (class)

```
package bank.management.system;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class Signup2 extends JFrame implements ActionListener {

    JComboBox comboBox,comboBox2,comboBox3,comboBox4,comboBox5;

    JTextField textPan,textAddar; JRadioButton r1,r2, e1,e2;

    JButton next,cancel;

    String FormNo;

    Signup2(String FormNo){

        super("PAGE-2 APPLICATION FORM");

        this.FormNo=FormNo;

        getContentPane().setBackground(new Color(18, 32, 47)); // Deep Navy

        ImageIcon i1 = new ImageIcon(ClassLoader.getResource("icons/rupes.png"));
        Image i2 = i1.getImage().getScaledInstance(100, 100, Image.SCALE_DEFAULT);
        ImageIcon i3 = new ImageIcon(i2); JLabel image = new JLabel(i3);

        image.setBounds(10, 10, 70, 100);

        add(image);

        Color labelColor = new Color(220, 230, 255);

        Color fieldBg = new Color(245, 248, 255);

        JLabel label1 = new JLabel("APPLICATION FORM NO.");

        label1.setForeground(Color.WHITE);

        label1.setFont(new Font("AvantGarde", Font.BOLD, 38));

        label1.setBounds(300, 35, 600, 40);

        add(label1);

        JLabel L1 =new JLabel("PAGE 2");

        L1.setForeground(new Color(200,200,200));
```

```
L1.setBounds(535, 75, 100, 30);

L1.setFont(new Font("AvantGarde", Font.BOLD, 22));

add(L1);

JLabel L2 = new JLabel("ADDITIONAL DETAILS");

L2.setForeground(new Color(0, 123, 255)); // Highlight Blue

L2.setBounds(460, 105, 250, 30);

L2.setFont(new Font("AvantGarde", Font.BOLD, 22));

add(L2);

JLabel L3 = new JLabel("Religion");

L3.setForeground(new Color(220,220,220));

L3.setBounds(100, 190, 150, 30);

L3.setFont(new Font("AvantGarde", Font.BOLD, 22));

add(L3);

String religion[]={"Hindu","Muslim","Sikh","Christian","Other"};

comboBox= new JComboBox(religion);

comboBox.setBackground(new Color(240,240,240));

comboBox.setFont(new Font("AvantGarde", Font.BOLD, 14));

comboBox.setBounds(250, 190, 250, 30);

comboBox.setForeground(Color.BLACK);

add(comboBox);

JLabel L4 = new JLabel("Category");

L4.setForeground(new Color(220,220,220));

L4.setFont(new Font("AvantGarde", Font.BOLD, 20));

L4.setBounds(565, 190, 200, 30);

add(L4);

String category[]={"General","OBC","SC","ST","Other"};

comboBox2= new JComboBox(category);

comboBox2.setBackground(new Color(240,240,240));

comboBox2.setForeground(Color.BLACK);
```

```
comboBox2.setFont(new Font("AvantGarde", Font.BOLD, 14));
comboBox2.setBounds(765, 190, 250, 30);
add(comboBox2);

JLabel L5 = new JLabel("Income");
L5.setForeground(labelColor);
L5.setFont(new Font("AvantGarde", Font.BOLD, 20));
L5.setBounds(565, 240, 200, 30);
add(L5);

String income[]={"Null","<1,50,000","<2,50,000","<5,00,000","Upto 10,00,000","Above
10,00,000"};

comboBox3= new JComboBox(income);
comboBox3.setBackground(fieldBg);
comboBox3.setFont(new Font("AvantGarde",Font.BOLD,14));
comboBox3.setBounds(765,240,250,30);
add(comboBox3);

JLabel L6 = new JLabel("Education");
L6.setForeground(labelColor);
L6.setFont(new Font("AvantGarde", Font.BOLD, 20));
L6.setBounds(100, 240, 200, 30);
add(L6);

String education[]={"Non-Graduate","Graduate","Post-
Graduate","Doctrate","Intermediate","Highschool","Other"};

comboBox4= new JComboBox(education);
comboBox4.setBackground(fieldBg);
comboBox4.setFont(new Font("AvantGarde",Font.BOLD,14));
comboBox4.setBounds(250,240,250,30);
add(comboBox4);

JLabel L7 = new JLabel("Occupation");
L7.setForeground(labelColor);
```

```
L7.setFont(new Font("AvantGarde", Font.BOLD, 20));

L7.setBounds(100, 290, 200, 30);

add(L7);

String occupation[]={"Salaried ","Self-
Employed","Business","Student","Retired","Other"};

comboBox5= new JComboBox(occupation);

comboBox5.setBackground(fieldBg);

comboBox5.setFont(new Font("AvantGarde",Font.BOLD,14));

comboBox5.setBounds(250,290,250,30);

add(comboBox5);

JLabel L8 = new JLabel("Pan No");

L8.setForeground(labelColor);

L8.setFont(new Font("AvantGarde", Font.BOLD, 20));

L8.setBounds(565, 290, 200, 30);

add(L8);

textPan = new JTextField();

textPan.setBackground(fieldBg);

textPan.setForeground(Color.BLACK);

textPan.setFont(new Font("AvantGarde", Font.BOLD, 14));

textPan.setBounds(765, 290, 250, 30);

add(textPan);

JLabel L9 = new JLabel("Aadhar No");

L9.setForeground(labelColor);

L9.setFont(new Font("AvantGarde", Font.BOLD, 20));

L9.setBounds(100, 340, 200, 30);

add(L9);

textAddar = new JTextField();

textAddar.setBackground(fieldBg);

textAddar.setForeground(Color.BLACK);
```

```
textAddar.setFont(new Font("AvantGarde", Font.BOLD, 14));
textAddar.setBounds(250, 340, 250, 30);
add(textAddar);

JLabel L10 = new JLabel("Senior Citizen");
L10.setForeground(labelColor);
L10.setFont(new Font("AvantGarde", Font.BOLD, 20));
L10.setBounds(565, 340, 200, 30);
add(L10);

r1=new JRadioButton("Yes");
r1.setBackground(new Color(18,32,47)); // Highlight Blue
r1.setForeground(Color.WHITE);
r1.setFont(new Font("AvantGarde", Font.BOLD, 14));
r1.setBounds(765,340,70,30);
add(r1);

r2=new JRadioButton("No");
r2.setBackground(new Color(18,32,47));
r2.setForeground(Color.WHITE);
r2.setFont(new Font("AvantGarde", Font.BOLD, 14));
r2.setBounds(847,340,70,30);
add(r2);

ButtonGroup buttonGroup = new ButtonGroup();
buttonGroup.add(r1);
buttonGroup.add(r2);

JLabel L11 = new JLabel("Exiting A/C");
L11.setForeground(labelColor);
L11.setFont(new Font("AvantGarde", Font.BOLD, 20));
L11.setBounds(100, 390, 200, 30);
add(L11);

e1=new JRadioButton("Yes");
```

```
e1.setBackground(new Color(18,32,47));
e1.setForeground(Color.WHITE);
e1.setFont(new Font("AvantGarde", Font.BOLD, 14));
e1.setBounds(250,390,70,30);
add(e1);

e2=new JRadioButton("No");
e2.setBackground(new Color(18,32,47));
e2.setForeground(Color.WHITE);
e2.setFont(new Font("AvantGarde", Font.BOLD, 14));
e2.setBounds(332,390,70,30);
add(e2);

ButtonGroup buttonGroup1 = new ButtonGroup();
buttonGroup1.add(e1);
buttonGroup1.add(e2);

/* JLabel L12 = new JLabel("FORM NO");
L12.setForeground(labelColor);
L12.setFont(new Font("AvantGarde", Font.BOLD, 20));
L12.setBounds(565, 390, 200, 30);
add(L12); */

JLabel L13= new JLabel(FormNo);
L13.setForeground(Color.WHITE);
L13.setFont(new Font("AvantGarde", Font.BOLD, 38));
L13.setBounds(760, 35, 600, 40);
add(L13);

cancel = new JButton("CANCEL");
cancel.setFont(new Font("AvantGarde", Font.BOLD, 14));
cancel.setForeground(Color.WHITE);
cancel.setBackground(new Color(220,53,69));
cancel.setBounds(460, 500, 90, 30);
```

```
cancel.addActionListener(this);

add(cancel);

next = new JButton("NEXT");

next.setFont(new Font("AvantGarde", Font.BOLD, 14));

next.setForeground(Color.WHITE);

next.setBackground(new Color(0, 123, 255));

next.setBounds(600, 500, 90, 30);

next.addActionListener(this);

add(next);

setLayout(null);

setSize(1170, 650);

setLocation(100, 50);

//setUndecorated(true);

setVisible(true);
}

@Override public void actionPerformed(ActionEvent e)
{
    String Religion=(String) comboBox.getSelectedItemAt();
    String Category = (String) comboBox2.getSelectedItemAt();
    String Income = (String) comboBox3.getSelectedItemAt();
    String Education = (String) comboBox4.getSelectedItemAt();
    String Occupation = (String) comboBox5.getSelectedItemAt();
    String Pan = textPan.getText();
    String Aadhar = textAddar.getText();
    String SeniorCitizen="";
    if((r1.isSelected())){
        SeniorCitizen="Yes";
    } else if (r2.isSelected()) {
        SeniorCitizen="No";
    }
}
```

```
}

String ExistingAccount="";

if((e1.isSelected())){

    ExistingAccount="Yes";

} else if (e2.isSelected()) {

    ExistingAccount="No";

}

try{

    if (e.getSource()==cancel){

        System.exit(0);

    }else if (textPan.getText().equals("")||textAddar.getText().equals("")){

        UIManager.put("OptionPane.background",new Color(30,144,255) );

        UIManager.put("Panel.background",new Color(30,144,225));

        JOptionPane.showMessageDialog(null,"All fields are mandatory. Please ensure no
field is left blank.", "Validation Error",JOptionPane.INFORMATION_MESSAGE);

    } else if (e.getSource()==next){

        Conn c1=new Conn();

        String Q = "insert into Signuptwo
values("+FormNo+"",""+Religion+"",""+Category+"",""+Income+"",""+Education+"",""+Occupati
on+"",""+Pan+"",""+Aadhar+"",""+SeniorCitizen+"",""+ExistingAccount+"")";

        c1.statement.executeUpdate(Q);

        new Signup3(FormNo);

        setVisible(false);

    }

} catch (Exception E){

    E.printStackTrace();

}

}

public static void main(String[] args) {

    new Signup2(""); } }
```

Backend (Database Code)

```
create database banksystem;

use banksystem;

create table signup(
FormNo varchar(100),
Name varchar(100),
FatherName varchar(100),
Gender varchar(100),
BOB varchar(100),
MaritalStatus varchar(100),
Email varchar(100),
Address varchar(100),
City varchar(100),
PinCode varchar(100),
State varchar(100)
);

select * from signup;

truncate table signup;

create table signuptwo(
FormNo varchar(100),
Religion varchar(100),
Category varchar(100),
Income varchar(100),
Education varchar(100),
Occupation varchar(100),
Pan varchar(100),
Aadhar varchar(100),
SeniorCitizen varchar(100),
ExistingAccount varchar(100)
```

```
);  
select * from signuptwo;  
truncate table signuptwo;  
create table signuptthree(  
FormNo varchar(100),  
AccountType varchar(100),  
CardNo varchar(100),  
Pin varchar(100),  
Facility varchar(100)  
);  
select * from signuptthree;  
truncate table signuptthree;  
create table Login(  
FormNo varchar(100),  
CardNo varchar(100),  
Pin varchar(100)  
);  
select * from Login;  
truncate table Login;  
create table Deposit (  
Pin varchar(100),  
Data varchar(100),  
Type varchar(100),  
Amount varchar(100)  
);  
select * from Deposit;  
truncate table Deposit;
```